초음파센서 AVR 기반 자율주행차 프로젝트

2026.06.23 | 이규현

목차

1 프로젝트 개요

- 목적
- 주요 기능
- 구성 요소

2 프로젝트 설계 및 구현

- 조립 결과물 외관 사진
- 회로도
- FSM

3 동작 구현

- Manual Mode
- Auto Mode
- FND Display

4 코드 설명

- Manual Mode
- Auto Mode

5 결론

- 시사점

프로젝트 개요

1 목적

- 초음파 센서와 AVR 마이크로컨트롤러를 사용하여 자율주행과 장애물 회피가 가능한 시스템 구현

2 주요 기능

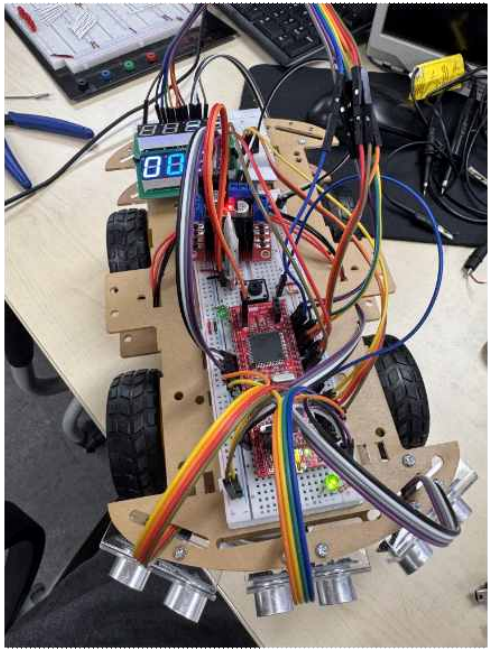
- 거리 측정 : 초음파 센서를 이용한 전방 및 측면 장애물 정밀 감지
- 주행 제어 : AVR을 이용한 DC 모터 속도 및 방향 제어
- 자율 주행 알고리즘 : 센서로부터 입력된 거리에 따른 실시간 회피 주행 및 경로 최적화

구성 요소

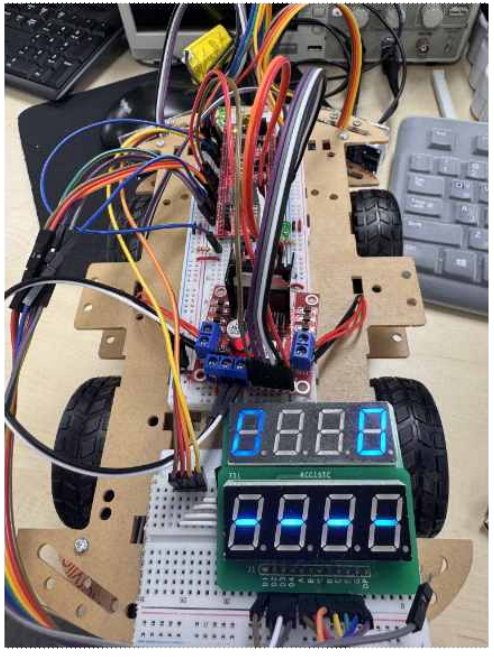
부품명	수량	부품명	수량
ATmega128A	1	초음파 센서 (HC-SR04)	3
모터 드라이버 (L298N)	1	푸시 버튼	1
DC모터	4	블루투스 모듈 (HC-06)	1
FND (7-Segment)	2	배터리	2

프로젝트 설계 및 구현

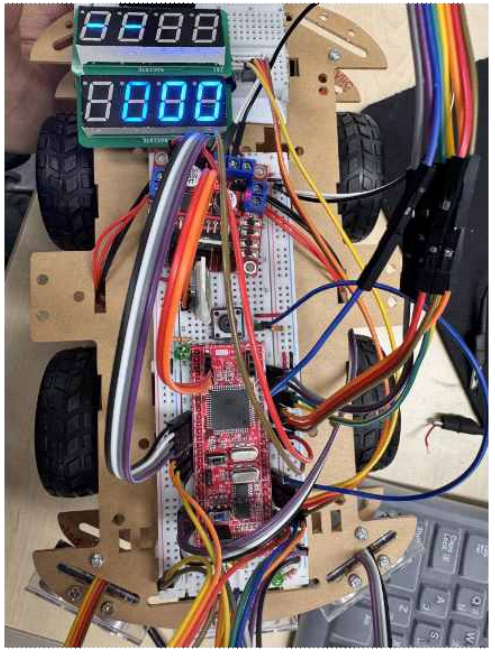
조립 결과물 외관 사진



정면



후면

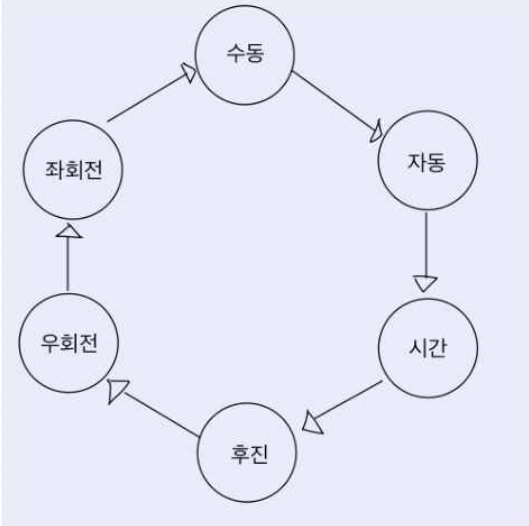


윗면

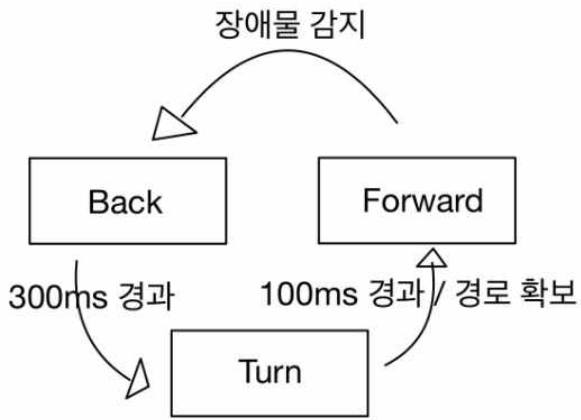


아랫면

프로젝트 설계 및 구현



모드 전환 FSM



자율 주행 FSM

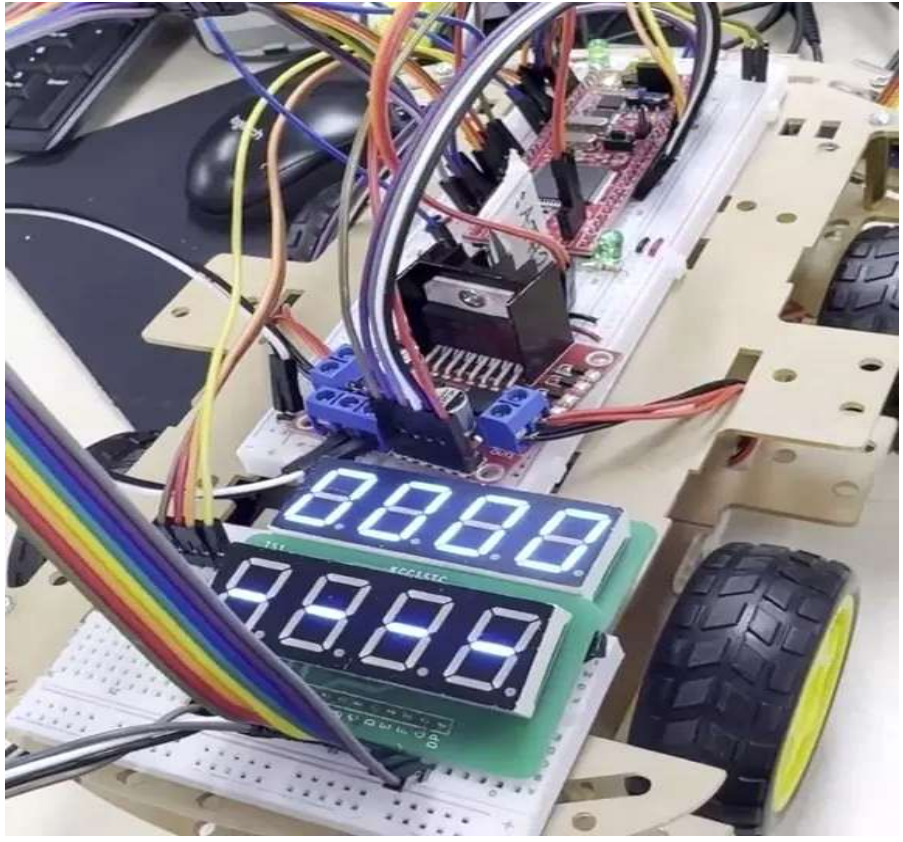
모드 전환 FSM

BUTTON 1을 토글할 때 마다 모드 전환

자율 주행 FSM

전진 중 장애물 감지 시
뒤로 후진 후 300MS 경과 시
여유있는 공간으로 회전
100MS 경과 또는 경로 확인 후 전진

동작 구현



Manual Mode

FND 1

1. 정지 상태 시 FND에 ---- 출력
2. 왼쪽 회전 시 FND에 E---출력
3. 오른쪽 회전 시 FND에 ---ㅋ 출력
4. 후진 시 FND에 ㅂㅂㅂㅂ 출력
5. 전진 시 FND에 ㄲㄲㄲㄲ 출력

MOTOR

1. 정지 상태 시 모터 동작 X
2. 왼쪽 회전 시 오른쪽 모터만 동작
3. 오른쪽 회전 시 왼쪽 모터만 동작
4. 후진 시 양쪽 모터 뒤로 동작
5. 전진 시 양쪽 모터 앞으로 동작

동작 구현



Auto Mode

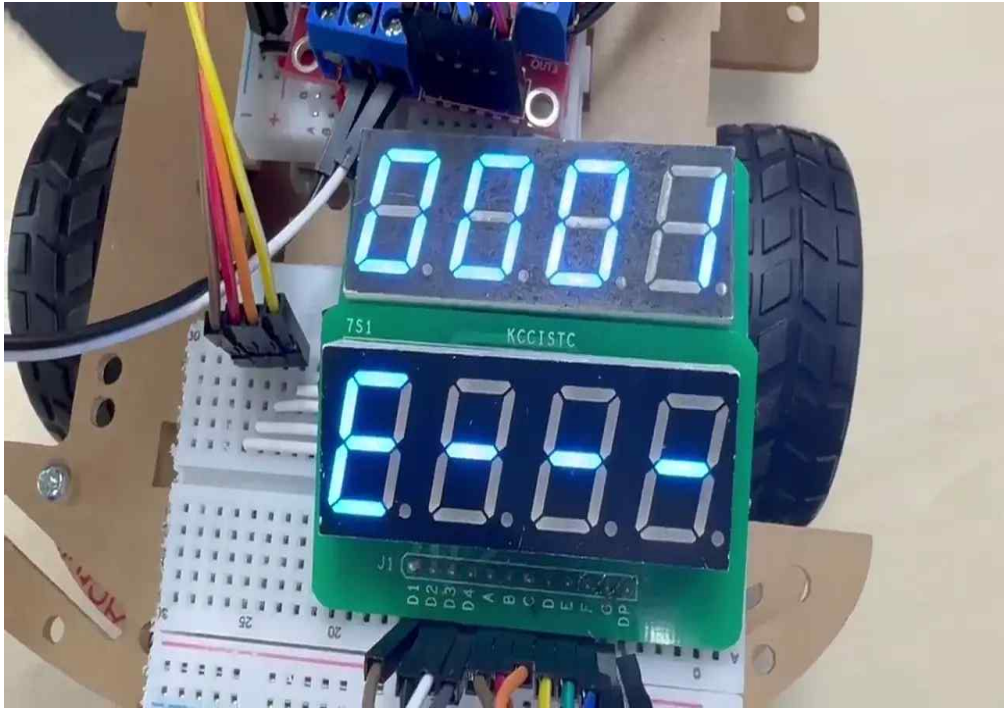
FND 1

1. 정지 상태 시 ---- 출력
2. 왼쪽 회전 시 E---출력
3. 오른쪽 회전 시 ---ㅋ 출력
4. 후진 시 ㄴㄴㄴㄴ 출력
5. 전진 시 ㄱㄱㄱㄱ 출력

MOTOR

1. 전진
   전방(M > 40), 좌측(L > 20), 우측(R > 20) 모두 안전거리를 확보했을 때만 실행
2. 전방 장애물 회피 (M <= 40):
   로봇이 정면에 장애물을 마주치면 300MS 동안 후진하여 공간을 확보
   그 후, 센서값 L과 R을 비교하여 더 넓은 공간이 있는 쪽으로 회전
3. 측면 장애물 조정 (L <= 20 또는 R <= 20):
   정면은 괜찮지만 벽에 너무 붙어 주행할 경우, 반대 방향으로 회전하여 경로를 수정

동작 구현



FND Display

FND 1

1. 버튼 1 토글 시 ---- 출력
2. 버튼 1 토글 시 ␣␣␣␣ 출력
3. 버튼 1 토글 시 E--- 출력
4. 버튼 1 토글 시 ---ㅋ 출력

FND 2

1. 버튼 1 토글 시 주행 시간 출력
2. 버튼 1 토글 시 후진 횟수 출력
3. 버튼 1 토글 시 우회전 횟수 출력
4. 버튼 1 토글 시 좌회전 횟수 출력

코드 설명

```
void manual_mode(void)
{
        switch(bt_data)
        {
                case 'F':
                case 'f':
                        forward(500);
                        prev_motor_state = 0;
                        break;
                case 'B':
                case 'b':
                        backward(500);
                        if (prev_motor_state != 1)
                        {
                           back_count++; prev_motor_state = 1;
                        }
                        break;
                case 'L':
                case 'l':
                        turn_left(700);
                        if (prev_motor_state != 3)
                        { left_count++; prev_motor_state = 3;
                        }
                        break;
                case 'R':
                case 'r':
                        turn_right(700);
                        if (prev_motor_state != 2)
                        {
                           right_count++; prev_motor_state = 2;
                        }
                        break;
                case 'S':
                case 's':
                        stop();
                        prev_motor_state = 0;
                        break;
                default: break;
```

코드 설명

```
void auto_mode(void)
{
        int L = (dist_left   <= 2) ? 400 : dist_left;
        int M = (dist_middle <= 2) ? 400 : dist_middle;
        int R = (dist_right  <= 2) ? 400 : dist_right;

        const int SPEED_FWD  = 300;
        const int SPEED_BACK = 300;
        const int SPEED_TURN = 420;

        const int THRESH_FRONT = 30;
        const int THRESH_SIDE  = 10;
        const int CLEAR_FRONT  = 40;
        const int CLEAR_SIDE   = 20;

        const int ERROR_THRESH = 1000;

        const uint32_t TIME_BACK     = 300;
        const uint32_t TIME_TURN_MIN = 150;
        const uint32_t TIME_TURN_MAX = 1000;

        int path_clear = (M > CLEAR_FRONT && M < ERROR_THRESH) &&
        (L > CLEAR_SIDE  && L < ERROR_THRESH)  &&
        (R > CLEAR_SIDE  && R < ERROR_THRESH);
```

```
switch (car_state)
        {
                case CAR_FORWARD:
                if (M >= ERROR_THRESH || L >= ERROR_THRESH || R >= ERROR_THRESH)
                {
                        if (L >= ERROR_THRESH) av_turn_dir = 1;
                        else if (R >= ERROR_THRESH) av_turn_dir = 0;
                        else av_turn_dir = (L >= R) ? 0 : 1;

                        backward(SPEED_BACK);
                        av_timer = msec_count;
                        car_state = CAR_BACK;
                        break;
                }

                if (M <= THRESH_FRONT)
                {
                        av_turn_dir = (L >= R) ? 0 : 1;
                        backward(SPEED_BACK);
                        av_timer = msec_count;
                        car_state = CAR_BACK;
                        break;
                }

                if (L <= THRESH_SIDE)
                {
                        av_turn_dir = 1;
                        backward(SPEED_BACK);
                        av_timer = msec_count;
                        car_state = CAR_BACK;
                        break;
                }

                if (R <= THRESH_SIDE)
                {
                        av_turn_dir = 0;
                        backward(SPEED_BACK);
                        av_timer = msec_count;
                        car_state = CAR_BACK;
                        break;
                }

                forward(SPEED_FWD);
                break;
```

코드 설명

```
case CAR_BACK:
backward(SPEED_BACK);

if (path_clear)
{
        car_state = CAR_FORWARD;
        forward(SPEED_FWD);
        break;
}

if (msec_count - av_timer >= TIME_BACK)
{
        if (av_turn_dir == 0)
        turn_left(SPEED_TURN);
        else
        turn_right(SPEED_TURN);
        av_timer = msec_count;
        car_state = CAR_TURN;
}
break;
```

```
case CAR_TURN:
        {
                uint32_t elapsed = msec_count - av_timer;

                if (elapsed < TIME_TURN_MIN)
                {
                        if (av_turn_dir == 0)
                        turn_left(SPEED_TURN);
                        else
                        turn_right(SPEED_TURN);
                        break;
                }

                if (path_clear || elapsed >= TIME_TURN_MAX)
                {
                        car_state = CAR_FORWARD;
                        forward(SPEED_FWD);
                        break;
                }

                if (av_turn_dir == 0)
                turn_left(SPEED_TURN);
                else
                turn_right(SPEED_TURN);
                break;
        }

        default:
        car_state = CAR_FORWARD;
        break;
}
update_auto_stats();
}
```

결과

시사점

문제점

- FND의 애니메이션 효과가 출력되는 것을 구현하지 못함
- AUTO MODE에서 장애물이 있을 때 앞뒤로 너무 자주 움직임

느낀점

모터 동작을 구현하면서 예외 처리가 중요하다는 생각이 듦
배선 문제도 생각보다 많이 발생된다는 것도 알게됨